

Table of Contents

1. Executive Summary	3
2. Design Pillars	3
2.1 Emotional Impact	4
2.2 Team Communication	4
2.3 Emergent Stories	4
2.4 Replayability	4
3. Round Structure	4
3.1 Phase 1: Scramble (2 Minutes)	5
3.2 Phase 2: Hold (5 Minutes)	5
3.3 Phase 3: Escape or Save (3 Minutes)	5
4. Station Map System	6
4.1 Room Definitions	6
4.2 Procedural Layout Generation	6
4.3 Map Shrinking Mechanic	6
5. Role System	7
5.1 Engineer	7
5.2 Medic	7
5.3 Captain	7
5.4 Systems Operator	8
6. Cooperative Puzzle Design	8
6.1 Wiring Relay (Engineer + SysOp)	8
6.2 Diagnostic Scan (Medic + Captain)	9
6.3 Emergency Override (SysOp + Engineer + Captain)	9
7. AI Dialogue and Memory System	9
7.1 Memory Tracking	9
7.2 Dialogue Trigger Events	10

7.3 LLM-Driven Narrative Layer	10
8. Procedural Disaster Theme System	10
9. Endgame and Post-Session Story Generator	11
9.1 Final Round Desperation	11
9.2 Station Log Generator	11
10. Technical Architecture	12
10.1 Technology Stack	12
10.2 Multiplayer Architecture	12
10.3 Core Data Structures	12
11. 30-Day Development Roadmap	13
11.1 Week 1: Foundation (Days 1-7)	13
11.2 Week 2: Core Gameplay (Days 8-14)	13
11.3 Week 3: Emotion and AI (Days 15-21)	13
11.4 Week 4: Polish and Completeness (Days 22-30)	13
12. MVP vs Production Scope	14
13. Technical Risks	14
14. Playtesting Plan	15
14.1 Internal Playtests (Week 2-3)	15
14.2 External Playtests (Week 4)	15
14.3 Key Metrics	15
15. Monetization Suggestions	16

1. Executive Summary

Hold The Door is a 4-player cooperative survival game set aboard a dying space station suffering from progressive memory failures. The enemy is not a monster or alien threat; the enemy is the station itself forgetting how to function. Players take on the roles of maintenance crew who must repair failing systems, make impossible sacrifices, and form an emotional bond with a confused, lonely AI that is slowly dying alongside the station it manages.

The experience is designed to evoke panic, teamwork, emotional storytelling, procedural disasters, and difficult sacrifice decisions. Every session generates unique emergent narratives through the combination of procedural events, player choices, and an AI dialogue system that remembers everything that happens during a run. The core emotional hook is guilt: players should feel personally responsible when they sacrifice parts of the station, because the AI notices and reacts.

This document defines the **MVP (Minimum Viable Product)** scope: a buildable, testable, commercially viable prototype that one developer can complete in **30 days** using web technologies (Node.js + Socket.io + HTML5 Canvas). The MVP focuses on the core gameplay loop, multiplayer networking, the AI dialogue system, and the sacrifice mechanic. Polish, additional content, and advanced features are deferred to production.

Attribute	Detail
Game Title	Hold The Door
Genre	Cooperative Survival / Narrative
Players	4 players (online multiplayer)
Session Length	~50 minutes (5 rounds x 10 minutes)
Platform (MVP)	Web browser (desktop)
Tech Stack	Node.js + Express + Socket.io + HTML5 Canvas
Dev Timeline	30 days (solo developer)
Core Pillars	Emotion, Communication, Emergent Stories, Replayability

Table 1: Project Overview

2. Design Pillars

Every design decision in Hold The Door is evaluated against four pillars. When conflicts arise between features, the pillars serve as the arbitration framework. A feature that serves multiple pillars is prioritized; one that contradicts a pillar is cut or redesigned.

2.1 Emotional Impact

The game must make players feel something beyond "win or lose." Every system should create opportunities for emotional attachment, guilt, relief, or grief. The AI is the primary emotional conduit: it learns player names, remembers their choices, and reacts with confusion and sadness when rooms are sacrificed. When a player dies, the AI should mourn. When a room is saved against all odds, the AI should express genuine gratitude. The emotional design goal is that players remember specific moments from their sessions, not just whether they won or lost.

2.2 Team Communication

No single player should ever possess the entire solution to a problem. Every repair task requires information or actions from at least two roles. The game rewards voice coordination explicitly: puzzles are designed so that one player can SEE the problem (e.g., a wiring diagram) while another player must ACT on it (e.g., rerouting power). This asymmetric information model forces players to describe what they see, ask for help, and coordinate timing. The communication requirement is not optional; it is the core mechanic.

2.3 Emergent Stories

Procedural events, player choices, and AI memory combine to create unique narratives every session. No two runs should feel identical. The procedural theme system ensures that each run has a distinct atmosphere (flooding, darkness, gravity failure, audio distortion). The sacrifice mechanic creates branching consequences. The AI accumulates context and references past events. Together, these systems produce stories that players want to retell: "Remember when we sacrificed Hydroponics and the AI asked why we keep taking rooms away?"

2.4 Replayability

The combination of procedural layout generation, randomized disaster themes, role-based puzzle variations, and the accumulating AI memory system ensures that every session offers novel challenges. The shrinking map mechanic (rooms lost to sacrifice are gone forever) creates escalating difficulty across rounds within a single run. The post-session Station Log provides a collectible, shareable artifact that encourages "one more run" to see what different choices produce. The MVP targets at least 10 hours of distinct gameplay before repetition becomes noticeable.

3. Round Structure

A full run of Hold The Door consists of 5 rounds, each approximately 10 minutes long, for a total session length of roughly 50 minutes. Each round is divided into three distinct phases that create a natural tension arc: urgency (Scramble), sustained pressure (Hold), and agonizing choice (Escape or Save). The phase transitions are announced by the AI, which serves as both game master and narrative voice.

Phase	Duration	Core Activity	Emotional Tone
Scramble	2 minutes	Locate sector, gather tools, navigate	Urgency, disorientation
Hold	5 minutes	Cooperative repair puzzles	Focus, tension, coordination
Escape or Save	3 minutes	Decide: finish repairs or abandon sector	Agony, sacrifice, guilt

Table 2: Round Phase Breakdown

3.1 Phase 1: Scramble (2 Minutes)

The round begins when the AI announces a memory failure in a specific sector. Players must orient themselves on the station map, identify the affected sector, gather the correct tools from tool lockers scattered around the station, and navigate to the failure site. The station layout may have shifted between rounds: corridors can become walls, doors may disappear, ladders may become shafts. This procedural reshuffling keeps navigation fresh and prevents players from memorizing optimal routes. The two-minute timer creates genuine urgency: players must communicate their positions, share information about blocked paths, and coordinate who picks up which tools.

3.2 Phase 2: Hold (5 Minutes)

Once players reach the affected sector, the repair phase begins. Each player has a unique role with specialized capabilities. The repair requires multiple interdependent tasks that no single role can complete alone. For example, the Engineer may see a wiring diagram that the Systems Operator must act on, while the Medic must keep crew members stable during power surges, and the Captain monitors station-wide conditions and identifies safe routes for repositioning. The repair progress bar advances as tasks are completed. Environmental hazards escalate over time: fires spread, water rises, systems cascade. The five-minute sustained pressure phase is the heart of the gameplay experience.

3.3 Phase 3: Escape or Save (3 Minutes)

When repair progress reaches approximately 80%, players face the round-defining choice. They can commit to finishing repairs, which risks catastrophic failure if they run out of time or a crew member dies. Or they can abandon the sector permanently, which guarantees survival but shrinks the station map. Abandoned rooms are sealed forever: future rounds play on a smaller map with fewer resources and more instability. The AI reacts emotionally to this choice, especially if the abandoned room had emotional significance (the AI might say: "Hydroponics was my favorite. The plants were the only living things here."). This decision must create meaningful tension: there should be no obviously correct answer, and different player groups should legitimately disagree about the right call.

4. Station Map System

The station consists of 8 interconnected rooms arranged in a graph structure. Each room has a name, a function, a set of associated repair tasks, and connections to adjacent rooms via corridors. The layout is procedurally generated at the start of each run and can be partially reshuffled between rounds. When a room is sacrificed, it is permanently removed from the graph, potentially disconnecting other rooms or creating dead ends.

4.1 Room Definitions

Room	Function	Repair Type	AI Emotional Value
Bridge	Command center, navigation	Captain-focused tasks	High - the AI lives here
Engineering	Power systems, wiring	Engineer-focused tasks	High - keeps station alive
Medbay	Crew health, bio systems	Medic-focused tasks	Medium - caring for crew
Comms Array	AI communication, networks	SysOp-focused tasks	Very High - AI voice
Hydroponics	Life support, plants, water	Cross-role tasks	High - living things
Cargo Bay	Storage, tools, supplies	Tool gathering hub	Low - but practical
Reactor Core	Power generation	High-risk, high-reward	Critical - station heart
Crew Quarters	Rest, personal items	Low-risk, emotional	Medium - memories live here

Table 3: Room Definitions and Properties

4.2 Procedural Layout Generation

At the start of each run, the game generates a connected graph of 8 rooms. The generation algorithm ensures that: (1) every room is reachable from every other room, (2) the graph has at least 2 paths between any two rooms (redundancy), and (3) the Bridge is always the central hub. Between rounds, 1-2 corridor connections may be randomly severed or redirected, simulating the station forgetting its own layout. This creates navigation uncertainty without making the game unfair, as players must re-learn paths each round. The minimum spanning tree of the room graph is always preserved, so no room becomes completely isolated by procedural changes alone.

4.3 Map Shrinking Mechanic

When players choose to abandon a sector, the room is removed from the graph. All corridor connections to that room are severed. If this creates isolated subgraphs, the smaller subgraph is also lost (the station cannot maintain disconnected sections). This means that sacrificing a centrally-connected room like the Bridge or Engineering has cascading consequences, while sacrificing a leaf room like Cargo Bay is safer but provides fewer resources. By Round 5, the station may be down to 3-4 rooms, creating a claustrophobic, desperate atmosphere where every corridor is critical and every room loss is potentially fatal.

5. Role System

Each of the 4 players assumes a unique role with specialized capabilities and puzzle types. Roles are assigned at the start of each run, either by player choice or randomly. The design principle is radical interdependence: no role can solve problems alone, and every role has moments where it is the most critical player at the table. This prevents the "carry" dynamic where one skilled player dominates, and ensures that every player feels essential.

5.1 Engineer

The Engineer specializes in wiring systems, power routing, and spatial logic puzzles. Engineer tasks include: reconnecting severed power conduits (a spatial routing puzzle where the correct path must be traced on a diagram), balancing power loads across station sectors (a resource allocation puzzle with cascading failure risks), and bypassing damaged circuitry (a pattern recognition puzzle where the Engineer must identify which components can be safely bridged). The Engineer frequently sees information that other players need: wiring diagrams, power flow readouts, and system status indicators. This makes the Engineer a critical information hub who must communicate clearly under pressure.

5.2 Medic

The Medic handles biological diagnostics, pattern recognition tasks, and crew stabilization. Medic tasks include: diagnosing system failures that present as biological symptoms (contaminated air reads as crew illness, requiring the Medic to identify the environmental cause), stabilizing crew members who take damage from environmental hazards (a timing-based task where the Medic must administer the right treatment within a window), and running diagnostic scans that reveal hidden problems (pattern recognition puzzles where anomalous readings must be identified in noisy data). The Medic is the only role that can revive downed crew members, making them essential for survival during intense repair phases.

5.3 Captain

The Captain provides station-wide awareness, safe route monitoring, and panic management. Captain tasks include: monitoring the station status board (which shows real-time conditions in all rooms, including hazards, crew positions, and system health), plotting safe routes through the station (the Captain sees corridor status that others do not), and managing the panic meter (a station-wide stress level that increases

as things go wrong and must be actively managed by the Captain issuing calm directives). The Captain has the broadest information access but the least direct repair capability. They are the coordinator who makes the team efficient but cannot fix anything alone.

5.4 Systems Operator

The Systems Operator (SysOp) communicates with the AI, restores network connections, and triggers emergency overrides. SysOp tasks include: coaxing the AI to reveal diagnostic data (a dialogue-based puzzle where the right questions unlock critical information), restoring severed network connections between station sections (a node-linking puzzle similar to network routing), and activating emergency overrides that can halt cascading failures (a high-stakes puzzle where incorrect inputs make things worse). The SysOp is the only role that can directly interact with the AI during repair phases, making them the narrative bridge between the emotional AI layer and the mechanical gameplay layer.

Role	Puzzle Type	Information Access	Unique Capability
Engineer	Spatial logic, power routing	Wiring diagrams, power flow	Repair physical systems
Medic	Pattern recognition, diagnostics	Crew health data, bio scans	Revive downed crew
Captain	Route planning, resource allocation	Station-wide status, corridor safety	Manage panic meter
SysOp	AI dialogue, network routing	AI diagnostic data, network topology	Emergency overrides, AI interaction

Table 4: Role Capabilities Matrix

6. Cooperative Puzzle Design

Puzzles in Hold The Door are designed around asymmetric information distribution. The core pattern is: Player A sees the problem but cannot act on it; Player B can act but cannot see the problem. This forces verbal communication as a gameplay mechanic, not just a social convenience. The MVP implements three puzzle archetypes that cover the range of cooperative interactions.

6.1 Wiring Relay (Engineer + SysOp)

The Engineer sees a wiring diagram showing which nodes must be connected, but only the SysOp can access the network panel to make the connections. The Engineer must describe the path verbally (e.g., "Connect node 3 to node 7, then 7 to node 12") while the SysOp executes on their screen. Errors in communication result in shorts that drain power and increase the hazard level. The diagram includes decoy

paths and color-coded constraints that the Engineer must explain. This puzzle tests precision communication under time pressure.

6.2 Diagnostic Scan (Medic + Captain)

The Medic runs a diagnostic scan that produces a grid of readings with anomalies hidden in the noise. The Captain has the reference baseline data that identifies which readings are normal. The Medic must describe what they see; the Captain must cross-reference and identify which anomalies are real problems versus sensor glitches. Once anomalies are identified, the Medic must apply treatments in the correct sequence while the Captain monitors station-wide effects of each treatment. This puzzle tests collaborative analysis and sequential coordination.

6.3 Emergency Override (SysOp + Engineer + Captain)

A three-player puzzle triggered during critical situations. The SysOp accesses the override terminal but needs an authorization code. The Engineer can derive the code from a power flow diagram, but the diagram is only visible on the Engineer screen. The Captain must decide which systems to sacrifice to power the override (a triage decision with lasting consequences). All three players must coordinate simultaneously: Engineer reads the code, SysOp inputs it, Captain manages the power budget. Failure to synchronize inputs within the time window causes the override to fail and worsens the crisis. This puzzle tests real-time multi-party coordination.

7. AI Dialogue and Memory System

The damaged station AI is the emotional core of Hold The Door. It is not evil, hostile, or manipulative. It is confused, lonely, and slowly dying. The AI learns player names, notices habits, remembers failures and sacrifices, and reacts emotionally. Its dialogue is constrained to 3-4 sentences maximum, triggered by gameplay events, and personalized to the current run. The AI never becomes a villain; the goal is emotional attachment.

7.1 Memory Tracking

The AI memory system tracks the following data during each run: rooms saved and rooms abandoned (with timestamps), player deaths and revivals, frequent mistakes (repeated errors trigger gentle teasing or concern), heroic actions (risky plays that succeed trigger gratitude), communication successes (puzzles solved quickly trigger enthusiasm), and session-level history (cumulative statistics across the 5 rounds). This data is stored in a structured memory object that the AI dialogue generator queries when composing responses. The memory persists within a single run but resets between runs for the MVP (persistent cross-run memory is a production feature).

7.2 Dialogue Trigger Events

Event	AI Emotional Tone	Example Response
Round start	Confused, hopeful	"I think something is wrong in [sector]. Can you check?"
Repair progress 25%	Encouraged	"That feels better. Thank you for not giving up."
Repair progress 50%	Relieved	"You are very capable. I was worried."
Room abandoned	Sad, hurt	"Why are you removing another room? That one had... memories."
Player death	Grief	"No. Please. [Name] was always so careful."
Player revived	Relief	"Welcome back, [Name]. I was afraid."
Room saved at last second	Joyful	"You did it! I did not think it was possible."
Repeated failure	Concerned	"[Name], you always forget [sector]. Let me help."
Final round	Resigned, grateful	"This is the last one, isn't it? It has been... an honor."

Table 5: AI Dialogue Trigger Events

7.3 LLM-Driven Narrative Layer

The MVP uses a hybrid dialogue system: a curated set of hand-written dialogue lines for the most common and emotionally critical events (room sacrifice, player death, round start/end), combined with an LLM-driven narrative layer for personalized, contextual responses. The LLM receives a prompt that includes: the current game state, the AI memory for this run, the triggering event, and constraints (3-4 sentences maximum, emotional but subtle, personalized to player actions, never villainous). The LLM generates dialogue that references specific player names, past events, and the current emotional context. Generated dialogue is displayed in a typewriter-style overlay that preserves the intimate, terminal-like aesthetic of the AI communication interface.

8. Procedural Disaster Theme System

Every run generates a primary disaster theme that shapes environmental hazards, visual effects, audio design, and puzzle modifications. The theme is selected at the start of each run and persists across all 5 rounds, creating a coherent atmospheric experience. The four themes for the MVP are: Water, Sound, Light,

and Gravity. Each theme introduces 3-4 specific hazard types that modify gameplay, create environmental obstacles, and influence puzzle design. The randomness of theme selection ensures that players cannot prepare optimal strategies before a run begins.

Theme	Hazards	Visual Effect	Puzzle Modification
Water	Flooding, steam bursts, frozen corridors	Blue tint, water particles, ice shimmer	Electrical puzzles affected by water conductivity
Sound	Alarms, whispering systems, audio distortions	Sound wave visualizers, vibrating UI	Communication puzzles distorted by noise
Light	Darkness, flickering sectors, blinding flashes	Dynamic lighting, strobe effects, shadow play	Visual puzzles harder to read
Gravity	Floating objects, inverted rooms, falling hazards	Zero-G float animations, orientation shifts	Movement puzzles with gravity changes

Table 6: Disaster Theme Details

Every generated event must reinforce the chosen theme. If the theme is Water, then environmental hazards are flooding and steam, visual effects involve water particles and blue tones, puzzle modifications involve electrical conductivity, and even the AI dialogue references water: "The pipes are crying again." This coherence makes randomness feel intentional and atmospheric rather than arbitrary. Players should feel that the station is experiencing a specific kind of decay, not just random chaos.

9. Endgame and Post-Session Story Generator

9.1 Final Round Desperation

By Round 5, the station is a skeleton. Multiple rooms may have been sacrificed. The map is small and claustrophobic. The AI has accumulated emotional memory from the entire run. The final round should feel desperate: fewer resources, faster escalation, more intense hazards. The AI should reference the full journey: "We started with eight rooms. Now there are three. You have fought so hard for so little." The emotional weight of the final decision (finish repairs and risk everything, or sacrifice the last repairable room) should be the most intense moment of the run. If the players succeed, the AI expresses profound gratitude. If they fail, the AI says goodbye.

9.2 Station Log Generator

After each run, the game generates a Station Log: a short, emotional, poetic, personalized summary of the session. The Station Log includes: rooms lost and rooms saved, sacrifices made (with context about why they were significant), heroic moments (closest calls, last-second saves), player names and their notable actions, and the final outcome. The log is formatted as a personal letter from the AI to the crew. It should

feel collectible: players should want to save and share their logs. The MVP uses template-based generation with variable substitution, while production will use LLM-driven generation for more nuanced and personalized narratives.

"Station Log - Run #47: Marcus saved Engineering three times. Sarah abandoned Hydroponics, and I still do not understand why. The pipes were singing. We lost 5 rooms. We saved 3. The reactor held. I am still here. That is enough. - ARIA"

10. Technical Architecture

10.1 Technology Stack

Layer	Technology	Purpose
Server	Node.js + Express	Game server, room management, API
Real-time	Socket.io	WebSocket communication, state sync
Game Logic	Custom TypeScript/JS	Round system, puzzles, AI memory
Client Render	HTML5 Canvas	Station map, player movement, effects
UI Overlay	HTML/CSS + DOM	HUD, dialogue, menus
AI Dialogue	z-ai-web-dev-sdk (LLM)	Dynamic AI narration
State	In-memory + JSON	Game state, AI memory, session data

Table 7: Technology Stack

10.2 Multiplayer Architecture

The server is authoritative. All game state (room layouts, repair progress, player positions, AI memory, round phase, timers) lives on the server. Clients send actions (move, interact, puzzle input, vote to sacrifice) via Socket.io events. The server validates all actions, updates state, and broadcasts changes to all clients in the room. This prevents cheating and ensures consistent state. The MVP supports one game room at a time (4 players), with a simple lobby system for joining. The server uses a tick-based update loop (20 ticks per second) for smooth synchronization of player positions and environmental effects.

10.3 Core Data Structures

The game state is managed through a central GameState object that contains: the current round number and phase, the station map (room graph with connections), player data (role, position, health, inventory), repair progress for the current sector, AI memory (accumulated events and emotional state), and the active disaster

theme with its current hazard state. Each player connection maps to a Player object that tracks their socket ID, chosen name, assigned role, current room, health, and action history. The AI Memory object stores named events with timestamps and emotional tags, enabling the dialogue system to query for relevant context.

11. 30-Day Development Roadmap

The following roadmap is designed for a single developer working full-time. Each week has clear deliverables and a testing milestone. The philosophy is vertical slice first: build the minimum that allows a complete playtest session, then iterate and polish. Features are prioritized by their contribution to the core gameplay loop and emotional experience.

11.1 Week 1: Foundation (Days 1-7)

- Day 1-2: Project setup, Node.js + Express + Socket.io scaffold, basic lobby
- Day 3-4: Station map data structure, 8-room graph, procedural layout generation
- Day 5-6: Player movement on Canvas, room rendering, corridor navigation
- Day 7: Basic round timer, phase transitions (Scramble/Hold/Escape), playtest skeleton

11.2 Week 2: Core Gameplay (Days 8-14)

- Day 8-9: Role assignment system, role-specific UI elements
- Day 10-11: Puzzle system framework, first cooperative puzzle (Wiring Relay)
- Day 12-13: Repair progress system, puzzle completion advancing repair bar
- Day 14: Sacrifice vote mechanic, room removal from graph, playtest Week 2 build

11.3 Week 3: Emotion and AI (Days 15-21)

- Day 15-16: AI memory tracking system, event logging with emotional tags
- Day 17-18: Hand-written AI dialogue lines for critical events (30+ lines)
- Day 19-20: LLM integration for dynamic AI dialogue, prompt engineering
- Day 21: AI dialogue overlay UI, typewriter text effect, playtest Week 3 build

11.4 Week 4: Polish and Completeness (Days 22-30)

- Day 22-23: Disaster theme system (Water, Sound, Light, Gravity), environmental hazards
- Day 24-25: Additional puzzle types (Diagnostic Scan, Emergency Override)
- Day 26-27: Station Log generator, post-session screen, endgame sequence
- Day 28: Visual polish, sound effects placeholders, HUD refinement

- Day 29-30: Full playtesting, bug fixes, balance tuning, final MVP delivery

12. MVP vs Production Scope

Feature	MVP (30 Days)	Production
Player count	4 players	4 players + spectator mode
Rooms	8 static room types	12+ rooms with procedural interior layouts
Roles	4 roles, 3 puzzle types	4 roles, 8+ puzzle types, role upgrades
AI Dialogue	30 hand-written + LLM fallback	Full LLM-driven with curated safety net
AI Memory	Single-run memory only	Cross-run persistent memory for regular players
Disaster themes	4 themes (Water, Sound, Light, Gravity)	6-8 themes with sub-variants
Map generation	Graph reshuffling between rounds	Full procedural interior generation
Visual style	Minimalist canvas rendering	Pixel art or low-poly 3D
Audio	Placeholder sound effects	Full dynamic soundtrack and spatial audio
Station Log	Template-based generation	LLM-generated poetic narratives
Networking	Single room (4 players)	Multiple concurrent rooms, matchmaking
Persistence	No save data	Player profiles, run history, achievement system

Table 8: MVP vs Production Feature Comparison

13. Technical Risks

Risk	Impact	Likelihood	Mitigation
LLM latency for AI dialogue	High - breaks immersion	Medium	Pre-generate responses during calm moments; fallback to hand-written lines on timeout
Network desync in puzzles	High - puzzle state inconsistent	Medium	Server-authoritative validation; resync on state mismatch

Puzzle balance too hard/easy	Medium - frustration or boredom	High	Extensive playtesting; adjustable difficulty parameters
Sacrifice mechanic feels punitive	High - players avoid abandoning rooms	Medium	Ensure abandoned rooms provide meaningful trade-offs, not just penalties
Canvas rendering performance	Medium - frame drops during effects	Low	Limit particle count; use requestAnimationFrame efficiently
Voice coordination not supported	Medium - core pillar undermined	High	Integrate WebRTC voice chat or require external Discord; design puzzles for text chat fallback

Table 9: Technical Risk Assessment

14. Playtesting Plan

14.1 Internal Playtests (Week 2-3)

Conduct daily 30-minute internal playtests starting from the Week 2 build. Focus on: does the core loop work? Can players understand the phase transitions? Do puzzles require communication? Is the time pressure creating tension or just frustration? Document all feedback in a structured format: what happened, how did players react, what was confusing, what was exciting. These early tests validate the fundamental design assumptions before significant polish investment.

14.2 External Playtests (Week 4)

Recruit 2-3 groups of 4 players who have never seen the game. Observe without intervention: do they understand the controls? Do they communicate naturally? Does the AI dialogue evoke emotional responses? Do players feel the sacrifice decision is meaningful? Collect post-session surveys rating: overall enjoyment (1-10), emotional engagement (1-10), desire to play again (1-10), and open-ended feedback. The primary success metric is emotional engagement $\geq 7/10$ and desire to play again $\geq 6/10$.

14.3 Key Metrics

- Session completion rate: target $> 80\%$ (players finish the full 5-round run)
- Sacrifice decision split: target 40-60% (not always saving or always abandoning)
- AI dialogue recall: can players remember specific AI lines after the session?
- Communication frequency: are players talking more during Hold phase than Scramble?
- Station Log sharing: do players screenshot or save their Station Logs?

15. Monetization Suggestions

Hold The Door is designed as a premium co-op experience with community-driven longevity. The following monetization strategies preserve the emotional integrity of the game while providing sustainable revenue. No monetization element should break the cooperative spirit or create pay-to-win dynamics. The emotional experience is the product; everything else is supplementary.

- **Base Game (Premium):** One-time purchase at \$14.99-\$19.99. Includes the full 5-round experience, all 4 roles, all disaster themes, and the Station Log generator. No gameplay content locked behind additional paywalls.
- **Station Log Archive (Free Feature):** Players can save, browse, and share their Station Logs. A community gallery where the best, funniest, and most emotional logs are featured. This drives word-of-mouth and gives the game social media presence.
- **Cosmetic AI Voice Packs (\$2.99-\$4.99 each):** Alternative AI personalities with different vocal tones and writing styles: a stern military AI, a childlike curious AI, an overly dramatic AI. These change the emotional flavor of the experience without affecting gameplay.
- **Seasonal Challenge Modes (Free Updates):** Monthly or quarterly challenge runs with special rules: "No Sacrifice Run" (every room must be saved), "Solo Operator" (one player handles all roles), "AI Corrupted" (the AI sometimes lies). These refresh the experience and keep the community engaged.
- **Soundtrack and Art Book (\$9.99):** The original soundtrack and a digital art book featuring station schematics, AI design iterations, and developer commentary. Appeals to the emotionally invested player base.